Gist: Final Version of the Product

Technical report: scope alignment, technical functionality, testing, and code quality

Student: Elvis Preye Kerebi
Course: Mission Capstone, African Leadership University
Repository: github.com/thepreakerebi/gist
Walkthrough video: youtu.be/Q_6_BG6ncAk

1 Overview

Gist is a training-free compression layer that sits in front of an omni-modal language model. Given a long video and a question, it decides which few frames and audio windows the model needs, *before* the model's vision and audio encoders run. Every published audiovisual compressor reviewed in the proposal prunes tokens after the encoders, so it pays the full encoder cost and the uncompressed peak memory. Gist instead encodes only what survives selection. Each answer comes with a list of evidence items: a timestamp, a modality, the transcript or on-screen text, a playable clip, and the reason the item was kept. The product has three surfaces: a Python library and command-line tool, a FastAPI HTTP service, and a Next.js web application with a video library and a chat-style question interface.

2 Scope alignment and feature validation

The approved proposal set three objectives. Objective 1 was to review the literature and characterise baselines. Objective 2 was to implement five components that can each be switched off for ablation and are each covered by tests. Objective 3 was to verify, against pre-registered thresholds, that accuracy is preserved while computation falls. Table 1 maps each planned feature to its status. The full version, with file-level evidence, is `docs/scope-alignment.md` in the repository.

Planned feature	Intended purpose	Status	Evidence
Media ingestion and candidates	Process each video once; later questions reuse the result	**Completed**	`media/`, `library/ingest.py`; 128 frames and 30 s audio windows per video
Cross-modal contrastive saliency	Score candidates against the question without the answering model	**Completed**	CLIP, CLAP and Whisper adapters; speech/sound dispatcher; CLAP made deterministic
Joint z-score MMR with temporal kernel	Compare sight and sound fairly; avoid picking the same moment twice	**Completed**	`core/scoring.py`, `core/compressor.py`; removing it costs 45.7 points
Tail merging	Summarise low-ranked near-duplicates instead of dropping them	**Completed**	`core/tail_merging.py`; off by default, unused in reported results
Query decomposition and intent	Route each part of a compound question to its modality	**Completed**	Rule-based, `core/decomposition.py`, `core/query_intent.py`
Confidence-routed adaptive budget	Spend more evidence only when unsure	**Completed**	6 → 12 → 24 item ladder; `tests/test_budget_ladder.py`
Inspectable evidence artefact	Let users verify answers; tell selection from reasoning failures	**Completed**	`gateway/evidence_package.py`; evidence panel in the web app (Figure 1)
Measurement harness and baselines	Quantify accuracy, cost and grounding with intervals	**Completed**	`eval/`; uniform, visual-only, transcript-only, top-k, dense; `results/`
Unquantised 7B model on one 24 GB GPU	Show feasibility on consumer hardware	**Completed**	Qwen2.5-Omni-7B fp16 on RTX 4090, no out-of-memory failure
Post-encoder head-to-head	Compare with published training-free methods	**Partial**	OmniZip run; OmniScope built, not run; OmniPack reimplemented (no public code)
Measured wall clock and peak memory	Replace derived FLOPs with measurements	**Partial**	Harness in `results/efficiency-measured/`, not yet executed
Corpus of ~40 recordings, ~200 cases	Stable estimates per question type	**Partial**	7 hour-plus recordings, 35 cases; 51 Video-MME questions
Held-out split, run once	An estimate free of tuning	**Partial**	12 cases frozen in `data/eval/splits/`, deliberately unrun
Public web demonstrator	Let a reviewer try the system	**Partial**	Complete locally; deployment scheduled for the week before defense

VideoLLaMA2-7B second model; SigLIP scorer	Check generality; compare scorers	**Not done**	GPU time went to the head-to-head and ablations
MUSIC-AVQA, AVQA, WorldSense	Sound-event and cross-modal cases	**Not done**	Clips of 10 to 60 s give a long-video compressor nothing to compress

Table 1. Planned features against delivered status.

Justification of changes. The corpus shrank after an integrity check found that one "61-minute" source was a 12-minute film repeated five times, and Video-MME's long split stops at 60 minutes. The OmniScope run, the efficiency run and the held-out split are all built and costed at about $2.50 of GPU time in total. The held-out split is run last on purpose, so it stays free of tuning. A second answering model is lower priority because Gist never reads that model's internals. A user study was outside the proposal's four-month scope.

3 Technical functionality

1 Ingest (once) download, sample 128 frames + 30 s audio windows, transcribe, OCR, embed	**2 Score** CLIP for frames, CLAP for sound, Whisper + sentence embeddings for speech	**3 Normalise** z-score within each modality so the two scales compare	**4 Select** MMR: relevance minus closeness in time to items already chosen	**5 Adapt** escalate 6 → 12 → 24 items on weak-selection signals	**6 Answer** selected clips only, with timestamped evidence

Core algorithm. Scores from different models have different ranges: CLIP similarities fall around 0.25 to 0.32 and CLAP around 0.4 to 0.8. Merged raw, sound would almost always win. Gist therefore z-scores each modality separately. Selection then uses maximal marginal relevance (Carbonell and Goldstein, 1998), with one change. Redundancy is measured by distance in time through a Gaussian kernel, $\exp(-\Delta t^2/\sigma^2)$, not by content similarity, because the repetition to remove is neighbouring frames of the same moment. A frame and a sound from the same moment count as complementary, not redundant. Three presets pair budget size with diversity strength: aggressive (6 items, λ 0.66, σ 10 s), balanced (12, 0.72, 14 s) and conservative (24, 0.78, 18 s). The adaptive budget starts at aggressive and escalates when nothing is selected, the best relevance is below 0.15, a visual item is linked to audio that was not kept, or every item comes from one modality. It has at most three rungs, and a budget set by the caller is a ceiling.

Backend and frontend interaction. The web app (Next.js 16, React 19, shadcn) calls the FastAPI service. Adding a YouTube link starts ingestion in the background. Each stage writes its progress to Postgres, so a page reload shows the true state, and a failed ingestion is marked with a readable message and offers retry and remove. A question opens a chat thread. The API encodes only the question with CLIP and CLAP, and pgvector computes cosine similarity against every stored fingerprint in one SQL query, so later questions never re-encode the video. The selected clips are cut with FFmpeg, re-encoded to H.264/AAC (yt-dlp's AV1/Opus played as a black frame in Chrome), joined, and sent to TwelveLabs Pegasus 1.5. The answer, the evidence timeline and the playable clips return to the UI.

Data handling. Postgres holds only derived data: metadata, transcripts, 512-dimensional CLIP and CLAP embeddings, and chat history. Source video stays on disk, because one 61-minute source is 756 MB against about 1 MB of derived data, and Postgres cannot serve the byte ranges that video scrubbing needs. Every reported accuracy and cost figure comes from Qwen2.5-Omni-7B run offline. No measured claim comes from the hosted answerer.

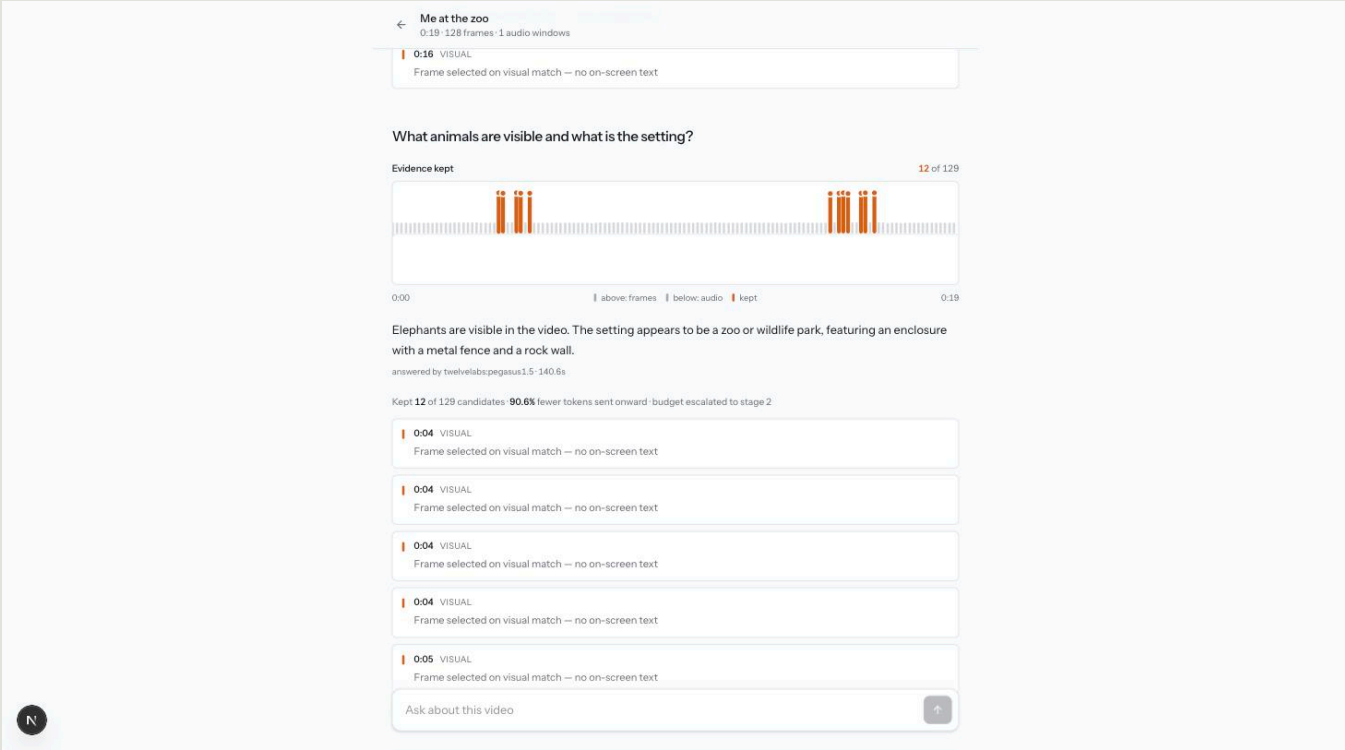


Figure 1. An answer with its evidence: what was kept, the token reduction, and each timestamped item the model was shown.

4 Testing approaches and results

Testing was layered so that the selection logic can be checked in seconds on any machine, while the model-dependent paths are checked separately. Table 2 lists the environments, and Table 3 summarises the four strategies.

Environment	Configuration	Used for
Development machine	Apple M4, 16 GB, macOS, Python 3.12.13, uv-pinned, CPU inference	Unit and integration tests, ingestion, scoring, web app
GPU pod	NVIDIA RTX 4090 24 GB, PyTorch 2.6.0 + CUDA 12.4, Transformers 5.13	Unquantised Qwen2.5-Omni-7B accuracy runs
OmniZip pod	Same GPU, Transformers 4.52.3 as pinned by OmniZip	Head-to-head comparison
Database	Neon Postgres 18.6, pgvector 0.8.6	Stored fingerprints, transcripts, chat history
Web client	Next.js on Bun, Google Chrome at 1440 × 900	Interface and functional testing

Table 2. Test and execution environments.

Strategy	Why and how	Result	Improvement made
Unit	Pins each selection behaviour, often a failure found in development. 74 files, 546 cases with pytest	538 passed, 8 model tests skipped by default, 77% line coverage (selector 81%, scoring 93%, tail merging 96%, presets 100%)	A slide shown at minutes 2 and 20 was merged into a fake minute-11 item; a hard temporal gate was added, and a test now requires distant lookalikes never to merge
Integration	Checks that parts work together: FastAPI test client against real endpoints; real CLIP, CLAP and Whisper loaded	546 passed with models loaded; stored-embedding scores equal live scores to 1e-5	The stored route had dropped CLIP's caption template and matched the live route on only 1 of the top 5; fixed and pinned. CLAP scores varied between runs (cosine 0.89 to 0.97); seeding made them identical
Functional / system	End-to-end in the web app on 36- and 49-minute recordings with varied question types	6 of 201 candidates kept, 96.9% fewer tokens, grounded answer; adaptive escalation to 12 of 201 on a visual question	Failed ingestions previously had no actions; retry and remove were added. One selection miss documented (below)
Validation / performance	Benchmark runs against the proposal's pre-registered thresholds, with paired bootstrap intervals (10,000 resamples)	See Table 4	Peak memory profiling built as the next run

Table 3. Testing strategies, results and follow-up changes.

Different inputs and error handling

Inputs tested include short trims (3 minutes), 36- to 96-minute recordings, speech, visual, sound and temporal-ordering questions, and compound questions. Errors are made visible and recoverable. A failed download or ingestion marks the row as failed with a message. If CLIP or CLAP cannot load, scoring falls back to lexical matching against transcripts and OCR. The GPU harness records out-of-memory as a result, not a crash. If the API is unreachable, the UI replays a recorded run. Two failures are reported, not hidden. Temporal-ordering questions failed in every condition, because scoring moments independently cannot express "before" or "after". Asked how a speaker handles failure in a 49-minute interview, Gist kept 6 of 226 items that missed the passage. The evidence list showed this as a selection failure: the web route ranks audio by CLAP alone, without the transcript-based dispatcher the research pipeline uses.

Performance and compatibility

Requirement (Objective 3)	Threshold	Measured	Outcome
Answer accuracy	Within 1 SE of full context	Gist 33.3% vs full 27.8% (n = 18, fp16); +5.6 pp [+0.0, +16.7]	**Met (parity)**
Encoded frames and audio windows	≥ 60% fewer	5.9 items/question vs 184 dense (96.8% fewer); vs uniform 8 + 4, 51% fewer	**Met vs dense**
Timestamp grounding	≥ 70% hit	86% of 35 curated cases (development split)	**Met**
Single 24 GB GPU	No out-of-memory	Qwen2.5-Omni-7B fp16 + all scorers on RTX 4090	**Met**

Table 4. Validation against the pre-registered thresholds.

Condition (n = 18)	Frames	GFLOPs*	Saving
Dense, 64 frames	64	313,345	n/a
Uniform, 8 frames	8	23,376	92.5%
Gist	2.7	9,082	97.1%

Table 5. Encoder cost. *Derived from measured counts and the published encoder architecture; 36.7% below uniform at equal accuracy.

Method (RTX 4090)	Accuracy	Could run on
Full context	6/18	90 s clips
OmniZip (post-encoder)	6/18	90 s clips (16.8 GB audio OOM)
Gist (pre-encoder)	6/18	Full-length videos

Table 6. Head-to-head on identical questions.

Ablation (35 curated cases). Full Gist passed 74.3%. Without hand-written coverage rules it passed 71.4%, without MMR diversity 28.6%, visual-only 45.7%, transcript-only 28.6%, and uniform sampling 14.3%. On 13 speech questions Gist passed 9 and visual-only passed none. On 16 visual questions both passed 15, so listening helps exactly where the answer is spoken.

Hardware and software. On the high end, the unquantised 7B model ran with all three scorers on a 24 GB consumer card where OmniZip could not run full-length input. On the low end, the whole product (API, ingestion, scoring and web app) runs CPU-only on a 16 GB laptop. A server needs about 4 GB of RAM, because the answering model is never deployed. Ingestion is the slow step on CPU: Whisper has no Apple Silicon GPU backend, so a one-hour video takes 10 to 20 minutes, with progress streamed to the UI. Model weights take about 2 minutes to load at cold start. After ingestion, a new question costs one text embedding per model and one SQL query. The software ran across three Transformers versions (4.52, 5.12, 5.13), CUDA and CPU PyTorch builds, and macOS and Linux. Browser testing covered Google Chrome only, and wall-clock time and peak memory have not yet been profiled. Both are stated as limitations.

5 Code quality and engineering practice

Object-oriented design. Every model sits behind an interface defined as a `typing.Protocol`: `VisualFrameScorer`, `AudioWindowScorer`, `AudioTranscriber`, `FrameOcr` and `LlmGateway`. Concrete classes such as `HuggingFaceClipFrameScorer`, `HuggingFaceClapAudioScorer` and `FasterWhisperTranscriber` implement them. The answering models (TwelveLabs, OpenAI, Claude, Hugging Face, Ollama, and an echo stub for tests) are interchangeable, so `GistCompressor` works the same whichever model answers. This polymorphism is what lets 538 tests exercise the selector without loading a model. Encapsulation: model loading, device placement and the CLAP seeding fix stay inside their adapters, and database access goes through a repository module. Requests, candidates, selections and metrics are typed Pydantic models and frozen dataclasses, validated at the API boundary.

Separation of concerns and modularity. The package is split by responsibility: `core` (selection logic, no model imports), `vision`, `audio`, `media`, `library`, `db`, `api`, `gateway` and `eval`. That is about 28,000 lines of source and 16,000 lines of tests. Each borrowed mechanism names its source in its module docstring (MMR, token merging, CLIP, CLAP, Whisper), and own work is marked as such. Dependencies are pinned in `uv.lock`, and the model stacks

are optional extras, so the core installs without PyTorch. Code is linted with Ruff. Of the 86 remaining findings, 81 are line-length warnings. The repository has about 290 commits with descriptive messages. One known weakness is that `core/compressor.py` is about 1,700 lines and would read better split into selection, budget and coverage modules.

Algorithms and problem solving. Beyond the MMR variant and the budget ladder, the custom solutions include: a speech/sound dispatcher that compares each audio window with "a voice speaking" and "ambient sound" to choose transcript or acoustic scoring; a hard temporal gate in tail merging; similarity search inside the database with pgvector, so no question re-encodes the video; rule-based query decomposition; and paired bootstrap confidence intervals for every comparison. Where a rule was added after a failure, as with the temporal gate and the CLAP seed, the failure is recorded where the constant is defined.